

Scenario: Employee Database Management

A small company, *TechCorp*, maintains an employee database to store details about its employees. The database has a table named **Employees**, which contains the following columns:

id (INTEGER, Primary Key)

name (VARCHAR)

department (VARCHAR)

salary (INTEGER)

hire_date (DATE)

The HR manager needs to perform some basic SQL queries to manage and retrieve employee data.

1. Retrieve employees from the IT department who earn more than \$50,000, sorted by salary in descending order.
2. Count the number of employees in each department but only for departments where there are more than 5 employees.
3. Find the top 3 highest-paid employees in the company.
4. Retrieve employees hired after 2020 and sort them by department and then by hire date.
5. Update the salary of employees in the 'HR' department who earn less than \$40,000, increasing it by 10%.

1. **Retrieve employees from the IT department who earn more than \$50,000, sorted by salary in descending order.**

```
SELECT * FROM Employees
WHERE department = 'IT' AND salary > 50000
ORDER BY salary DESC;
```

2. **Count the number of employees in each department but only for departments where there are more than 5 employees.**

```
SELECT department, COUNT(*) AS total_employees
FROM Employees
GROUP BY department
HAVING COUNT(*) > 5;
```

3. **Find the top 3 highest-paid employees in the company.**

```
SELECT * FROM Employees
ORDER BY salary DESC
LIMIT 3;
```

4. **Retrieve employees hired after 2020 and sort them by department and then by hire date.**

```
SELECT * FROM Employees
WHERE hire_date > '2020-01-01'
ORDER BY department ASC, hire_date ASC;
```

5. **Update the salary of employees in the 'HR' department who earn less than \$40,000, increasing it by 10%.**

```
UPDATE Employees
SET salary = salary * 1.10
WHERE department = 'HR' AND salary < 40000;
```